REST Client API Testing Guide

E-Commerce Workflow: Variables, Request Chaining & Data Storage

VS Code Extension • March 2026 • **Version 1.0**

1. Introduction & Setup

The REST Client extension for VS Code lets you write, send, and chain HTTP requests directly in .http files. This guide focuses on building a complete e-commerce API test suite — from login to checkout — using variables to store and reuse tokens, IDs, and cart state across multiple requests.

1.1 Why Use REST Client for E-Commerce APIs?

E-commerce APIs require data to flow between requests. A login generates an auth token. That token is needed by every subsequent call. A product search returns a product ID used to add items to a cart. REST Client handles all of this natively through request variables and environment settings.

1.2 File Structure

Organize your .http files by domain to keep tests manageable:

```
project/
  api-tests/
    auth.http          # Login, refresh token, logout
    products.http      # Catalog, search, product details
    cart.http          # Add/remove/view cart items
    orders.http        # Checkout, payment, order status
    admin.http         # Create/update products (admin use)
    .env               # Shared secrets (do not commit!)
    variables.http     # Shared file-level variable definitions
```

TIP: Set the file language to HTTP (.http extension) so you get syntax highlighting, CodeLens, and auto-complete.

2. Environments & Variables

REST Client supports four variable types. Understanding them is critical for building reliable test chains. The table below summarizes when to use each:

Variable Type	Use Case & Scope
Environment	Base URLs, API keys, env-specific secrets. Defined in VS Code settings. Available across all .http files.
File	Values constant within one .http file (e.g., endpoint path prefix). Defined with @name = value syntax.
Request	Captures a value from a previous response (e.g., auth token). Defined with # @name and referenced with {{name.response...}}.
Prompt	User-input variables requested at send time (e.g., OTP, one-time codes). Defined with # @prompt.
System	Built-in dynamic values: GUIDs, timestamps, random integers. Referenced as {{$guid}}, {{$timestamp}}, etc.

2.1 Environment Setup

Define environments in your VS Code settings.json. Use the $shared environment for variables that don't change across environments:

```
// settings.json
"rest-client.environmentVariables": {
  "$shared": {
    "apiVersion": "v1",
    "appName": "ShopApp"
  },
  "development": {
    "baseUrl": "http://localhost:3000",
    "adminToken": "{{$shared devAdminToken}}"
  },
  "staging": {
    "baseUrl": "https://staging.myshop.com",
    "adminToken": "{{$shared stgAdminToken}}"
  },
  "production": {
    "baseUrl": "https://api.myshop.com",
    "adminToken": "{{$shared prodAdminToken}}"
  }
}
```

NOTE: Switch environments using Ctrl+Alt+E (Cmd+Alt+E on macOS) or clicking the environment name in the status bar. All {{baseUrl}} references will automatically resolve to the selected environment.

2.2 File-Level Variables

Define reusable values at the top of your .http file. These are available to every request in the file:

```
// cart.http
@baseUrl     = {{baseUrl}}/api/{{apiVersion}}
@contentType = application/json
@currency    = USD

# These values are defined once and reused across all requests below
```

3. Authentication — Login & Token Storage

Authentication is the foundation of an e-commerce test chain. The login request returns an auth token and refresh token that all subsequent requests depend on. REST Client captures these automatically using request variables.

3.1 The Login Request

```
// auth.http
@baseUrl     = {{baseUrl}}/api/v1
@contentType = application/json

###############################################
# Step 1: Login — Captures auth + refresh tokens
###############################################

# @name login
POST {{baseUrl}}/auth/login HTTP/1.1
Content-Type: {{contentType}}

{
  "email":    "testuser@example.com",
  "password": "{{$processEnv USER_PASS}}"
}
```

SECURITY: Never hardcode passwords in .http files. Use {{$processEnv VARIABLE_NAME}} to read from your OS environment variables or {{$dotenv VARIABLE_NAME}} to read from a .env file.

3.2 Extracting Tokens from the Response

After sending the login request, REST Client stores the full response. You can now extract any field using JSONPath syntax:

```
# Extracting values from the login response:
```

```
# Auth token — stored in Authorization header
@authToken    = {{login.response.headers.Authorization}}

# OR if the token is in the response body (JSON)
@authToken    = {{login.response.body.$.data.accessToken}}
@refreshToken = {{login.response.body.$.data.refreshToken}}
@userId       = {{login.response.body.$.data.user.id}}
@userRole     = {{login.response.body.$.data.user.role}}

# Example response body that the above JSONPath targets:
# {
#   "data": {
#     "accessToken":  "eyJhbGciOi...",
#     "refreshToken": "dGhpcyBpcyB...",
#     "user": { "id": "usr_abc123", "role": "customer" }
#   }
# }
```

3.3 Refreshing the Access Token

```
###############################################
# Step 2: Refresh Token (run when token expires)
###############################################

# @name refreshAuth
POST {{baseUrl}}/auth/refresh HTTP/1.1
Content-Type: {{contentType}}

{
  "refreshToken": "{{refreshToken}}"
}

# After sending, update the authToken variable:
@authToken = {{refreshAuth.response.body.$.data.accessToken}}
```

4. Products — Browsing, Search & Capturing IDs

Product IDs are needed to add items to a cart, create orders, or trigger admin actions. Capture them from search or detail responses and store them as file variables.

4.1 Get All Products (with Filters)

```
// products.http
@baseUrl     = {{baseUrl}}/api/v1
```

```
@authToken   = {{login.response.body.$.data.accessToken}}
@contentType = application/json

###############################################
# Get Products — filtered, paginated
###############################################

# @name getProducts
GET {{baseUrl}}/products HTTP/1.1
Authorization: Bearer {{authToken}}
Accept: application/json

# Query params split across lines for readability:
    ?category=electronics
    &minPrice=100
    &maxPrice=1000
    &page=1
    &limit=20
    &sortBy=price
    &order=asc
```

4.2 Get Single Product & Capture ID

```
###############################################
# Get product detail & capture its ID + price
###############################################

# @name getProduct
GET {{baseUrl}}/products/prod_XYZ789 HTTP/1.1
Authorization: Bearer {{authToken}}
Accept: application/json

###

# Capture the product ID and price from the response
@productId    = {{getProduct.response.body.$.product.id}}
@productPrice = {{getProduct.response.body.$.product.price}}
@productSku   = {{getProduct.response.body.$.product.sku}}

# Now use {{productId}} in subsequent cart or order requests
```

4.3 Search Products

```
###############################################
# Search products — capture first result ID
###############################################

# @name searchProducts
GET {{baseUrl}}/products/search HTTP/1.1
```

```
Authorization: Bearer {{authToken}}

    ?q=wireless+headphones
    &inStock=true

# Capture first result from the array using JSONPath index
@firstProductId    = {{searchProducts.response.body.$.results[0].id}}
@firstProductPrice = {{searchProducts.response.body.$.results[0].price}}
```

TIP: You MUST send the named request (e.g., getProduct) before its response variables are available. The CodeLens tooltip shows the resolved value when you hover over a variable reference.

5. Cart — Add Items, Update & Review

The cart is the core of the e-commerce flow. It depends on the auth token (from login) and product IDs (from product requests). All three pieces of data are chained together here.

5.1 Create or Get Active Cart

```
// cart.http
@baseUrl     = {{baseUrl}}/api/v1
@authToken   = {{login.response.body.$.data.accessToken}}
@userId      = {{login.response.body.$.data.user.id}}
@contentType = application/json

###############################################
# Get or create active cart for current user
###############################################

# @name getCart
GET {{baseUrl}}/cart HTTP/1.1
Authorization: Bearer {{authToken}}
Accept: application/json

###

# Capture cart ID for use in add/remove/checkout
@cartId = {{getCart.response.body.$.cart.id}}
```

5.2 Add Item to Cart

```
###############################################
# Add a product to the cart
###############################################
```

```
# @name addToCart
POST {{baseUrl}}/cart/{{cartId}}/items HTTP/1.1
Authorization: Bearer {{authToken}}
Content-Type: {{contentType}}

{
  "productId": "{{productId}}",
  "sku":       "{{productSku}}",
  "quantity":  2,
  "currency":  "USD"
}

###

# Capture the cart item ID (needed to update/remove)
@cartItemId = {{addToCart.response.body.$.item.id}}
```

5.3 Update Item Quantity

```
##############################################
# Update quantity of a specific cart item
##############################################

# @name updateCartItem
PATCH {{baseUrl}}/cart/{{cartId}}/items/{{cartItemId}} HTTP/1.1
Authorization: Bearer {{authToken}}
Content-Type: {{contentType}}

{
  "quantity": 3
}
```

5.4 Remove Item & Clear Cart

```
##############################################
# Remove one item from cart
##############################################

DELETE {{baseUrl}}/cart/{{cartId}}/items/{{cartItemId}} HTTP/1.1
Authorization: Bearer {{authToken}}

###

##############################################
# Clear entire cart
##############################################

DELETE {{baseUrl}}/cart/{{cartId}} HTTP/1.1
```

```
Authorization: Bearer {{authToken}}
```

6. Orders — Checkout & Payment

The checkout flow uses the cartId, authToken, and optionally the userId — all captured from earlier requests. This is the final step of the e-commerce chain.

6.1 Create Order from Cart

```
// orders.http
@baseUrl      = {{baseUrl}}/api/v1
@authToken    = {{login.response.body.$.data.accessToken}}
@cartId       = {{getCart.response.body.$.cart.id}}
@userId       = {{login.response.body.$.data.user.id}}
@contentType  = application/json

###############################################
# Step 1: Create Order (checkout)
###############################################

# @name createOrder
POST {{baseUrl}}/orders HTTP/1.1
Authorization: Bearer {{authToken}}
Content-Type: {{contentType}}
X-Request-ID: {{$guid}}

{
  "cartId": "{{cartId}}",
  "userId": "{{userId}}",
  "shippingAddress": {
    "line1":   "42 Baker Street",
    "city":    "London",
    "country": "GB",
    "zip":     "NW1 6XE"
  },
  "billingAddress": {
    "sameAsShipping": true
  }
}

###

# Capture order ID and payment intent for next step
@orderId       = {{createOrder.response.body.$.order.id}}
@paymentIntent = {{createOrder.response.body.$.order.paymentIntentId}}
```

6.2 Process Payment

```
###############################################
# Step 2: Confirm Payment
###############################################

# @name confirmPayment
POST {{baseUrl}}/payments/confirm HTTP/1.1
Authorization: Bearer {{authToken}}
Content-Type: {{contentType}}

{
  "orderId":         "{{orderId}}",
  "paymentIntentId": "{{paymentIntent}}",
  "paymentMethod": {
    "type":   "card",
    "cardId": "card_test_4242424242424242"
  }
}

###

@paymentStatus = {{confirmPayment.response.body.$.payment.status}}
@receiptUrl    = {{confirmPayment.response.body.$.payment.receiptUrl}}
```

6.3 Get Order Status

```
###############################################
# Step 3: Confirm order is placed
###############################################

# @name getOrderStatus
GET {{baseUrl}}/orders/{{orderId}} HTTP/1.1
Authorization: Bearer {{authToken}}
Accept: application/json
```

7. Admin — Seeding Test Data

Before running tests, you often need products, categories, or users to already exist. Use admin API calls with pre-defined payloads to seed your test environment. This is especially useful in CI/CD pipelines.

7.1 Create a Product (Admin)

```
// admin.http
@baseUrl      = {{baseUrl}}/api/v1
```

```
@adminToken   = {{adminLogin.response.body.$.data.accessToken}}
@contentType  = application/json

###############################################
# Admin Login (separate from customer login)
###############################################

# @name adminLogin
POST {{baseUrl}}/auth/admin/login HTTP/1.1
Content-Type: {{contentType}}

{
  "email":    "admin@myshop.com",
  "password": "{{$processEnv ADMIN_PASS}}"
}

###

###############################################
# Create a new product
###############################################

# @name createProduct
POST {{baseUrl}}/admin/products HTTP/1.1
Authorization: Bearer {{adminToken}}
Content-Type: {{contentType}}

{
  "name":        "Wireless Headphones Pro X",
  "sku":         "WHP-PRO-X-{{$randomInt 1000 9999}}",
  "price":       299.99,
  "currency":    "USD",
  "category":    "electronics",
  "stock":       150,
  "description": "Premium noise-cancelling wireless headphones",
  "createdAt":   "{{$datetime iso8601}}"
}

@newProductId = {{createProduct.response.body.$.product.id}}
```

7.2 Create a Test User

```
###############################################
# Create test customer account
###############################################

# @name createTestUser
POST {{baseUrl}}/admin/users HTTP/1.1
Authorization: Bearer {{adminToken}}
Content-Type: {{contentType}}
```

```
{
  "email":     "test_{{$randomInt 100 999}}@example.com",
  "password":  "Test@1234",
  "role":      "customer",
  "firstName": "Test",
  "lastName":  "User",
  "createdAt": "{{$datetime iso8601}}"
}
```

8. Prompt Variables — Interactive Testing

Prompt variables pause execution and ask you to enter a value at send time. This is ideal for OTPs, one-time coupon codes, or any input that cannot be pre-defined.

```
###############################################
# Apply a coupon — prompts for code at send time
###############################################

# @prompt couponCode Enter your coupon code
# @prompt orderTotal Expected order total before discount

POST {{baseUrl}}/cart/{{cartId}}/coupon HTTP/1.1
Authorization: Bearer {{authToken}}
Content-Type: {{contentType}}

{
  "couponCode":   "{{couponCode}}",
  "orderTotal":   {{orderTotal}},
  "currency":     "USD"
}
```

9. Full End-to-End Chain — Quick Reference

The diagram below shows how data flows between requests. Each arrow represents a variable captured from a response and used in the next request:

```
  [1] auth.http
      POST /auth/login
          |
          +-- @authToken    --> Used in ALL subsequent requests
          +-- @refreshToken --> Used in POST /auth/refresh
          +-- @userId       --> Used in POST /orders
          |
```

```
[2] products.http
    GET /products/search
        |
        +-- @productId    --> Used in POST /cart/:id/items
        +-- @productSku   --> Used in POST /cart/:id/items
        +-- @productPrice --> Used for assertion / validation
        |
[3] cart.http
    GET /cart
        |
        +-- @cartId      --> Used in POST /orders
    POST /cart/:id/items
        |
        +-- @cartItemId  --> Used in PATCH/DELETE /cart/:id/items/:itemId
        |
[4] orders.http
    POST /orders
        |
        +-- @orderId        --> Used in POST /payments/confirm
        +-- @paymentIntent  --> Used in POST /payments/confirm
    POST /payments/confirm
        |
        +-- @paymentStatus  --> Verified via GET /orders/:id
```

10. Tips, Shortcuts & Best Practices

10.1 Keyboard Shortcuts

Action	Windows/Linux | macOS
Send Request	Ctrl+Alt+R | Cmd+Alt+R
Cancel Request	Ctrl+Alt+K | Cmd+Alt+K
Rerun Last Request	Ctrl+Alt+L | Cmd+Alt+L
View Request History	Ctrl+Alt+H | Cmd+Alt+H
Switch Environment	Ctrl+Alt+E | Cmd+Alt+E
Generate Code Snippet	Ctrl+Alt+C | Cmd+Alt+C
Navigate to Symbol	Ctrl+Shift+O | Cmd+Shift+O

10.2 Best Practices

- Always use {{$processEnv}} or {{$dotenv}} for passwords and secrets — never hardcode them.

- Name every request that produces data another request depends on using # @name.
- Send the named request first before referencing its response variables in other requests.
- Use $shared environment for API version, app name, and other universal constants.
- Use {{$guid}} for X-Request-ID headers to uniquely identify each test request.
- Use {{$randomInt 1000 9999}} in generated SKUs/emails to avoid conflicts on repeat runs.
- Use ### delimiters to separate requests in a single file — keep each file to one domain.
- Store the .env file in .gitignore — never commit credentials to source control.

10.3 Debugging Failed Variable References

If a variable like {{login.response.body.$.data.accessToken}} is sent as plain text instead of its value, check the following:

- The named request (# @name login) has been sent at least once this session.
- The JSONPath expression matches the actual response structure.
- Hover over the variable reference — VS Code shows the resolved value in a tooltip.
- The response status was 2xx — failed responses won't populate variables.

NOTE: Request variables are session-scoped. They reset when VS Code restarts. Re-send the login and other named requests after restarting to repopulate all variable values.

REST Client Extension — Official Documentation: https://github.com/Huachao/vscode-restclient